rag -- 2026-11-04

5 new paper(s) · topic: Retrieval-augmented generation: retrieval quality, grounding and faithfulness, evaluation benchmarks, and RAG system design.

Worth reading in full

[P5] QueryFlow: Multi-Hop Graph Retrieval with Adaptive Query Evolution -- P5 introduces a novel recurrent query-evolution mechanism that substantially improves multi-hop evidence retrieval and downstream answer quality while also cutting retrieval-plus-answer time by over 90 %, making it the most impactful advance for RAG system design in this set.

Themes

Several papers focus on improving or diagnosing the retrieval component of RAG systems. P2 shows that even tiny amounts of query-relevant corpus poisoning can collapse nDCG@10 by up to 60 % for BM25 and Hybrid retrievers, highlighting a severe robustness gap in current admission steps [P2]. In contrast, P5 tackles retrieval weakness from a different angle by adaptively evolving the query across graph hops, which yields 5-9 pp gains in Recall@10 and 6-12 pp improvements in downstream LLM accuracy over static baselines such as LinearRAG and GFM-RAG [P5]. Both studies expose retrieval as a bottleneck, but while P2 emphasizes adversarial failure modes, P5 proposes a constructive architectural fix.

Another strand addresses the cost and faithfulness of RAG pipelines. P1 introduces RAGLog, a reference-free fault-localization tool that flags stage-level errors with pooled F1 ~ 0.73 on natural runs and 0.84 on injected runs, doing so in about five seconds per run and at $0.005 cost, thereby offering a lightweight diagnostic alternative to heavyweight instrumentation [P1]. Meanwhile, P4 presents ResMem, a parametric-memory approach that compresses the external knowledge store by 94.7 % and lifts mean token F1 from 32.83 % to 34.48 % on five QA datasets, at the expense of only an 8.6 % latency increase, illustrating a trade-off between storage efficiency and modest accuracy gains [P4].

Evaluation breadth varies across the set. P3 applies a standard FAISS + BM25 pipeline to a financial domain, reporting high ROUGE scores and sub-second latency but without quantitative baselines, making its contribution more demonstrative than comparative. P1 and P2 provide concrete benchmark numbers (F1, nDCG@10, recall), while P5 supplies detailed recall and accuracy metrics across four multi-hop QA benchmarks, and P4 reports token-level F1 and hallucination detection accuracy. The collection therefore reveals a split between papers that propose new robustness or efficiency metrics (P1, P2, P4) and those that showcase end-to-end system performance on niche domains (P3, P5).

4 theme sentence(s) checked against the paper summaries.

At a glance

[P1] From Logs to Faults: Reference-Free Fault Localization for Retrieval-Augmented Generation Pipelines · relevance 0.47
RAGLog is a reference-free tool that localizes faults in Retrieval-Augmented Generation pipelines from execution logs, achieving pooled F1 ~ 0.73 on natural runs and ~ 0.84 on injected runs with low latency and cost.

[P2] Reliability Stress Testing of RAG Retrieval under Query-Relevant Corpus Poisoning · relevance 0.46
Query-aware corpus poisoning can dramatically degrade RAG retrieval quality, achieving near-perfect target exposure while dropping nDCG@10 by up to 60% even with a tiny fraction of poisoned documents.

[P3] Enhancing Stock Market Information Retrieval with Retrieval-Augmented Generation · relevance 0.43
The paper presents a Retrieval-Augmented Generation (RAG) system for stock-market queries that

achieves high ROUGE scores and low latency, demonstrating promising accuracy and speed for financial information retrieval.

[P4] From Lookups to Lessons: Turning Retrieved Knowledge into Parametric Memory · relevance 0.41
ResMem distills a retrieval-augmented teacher into an additive logit residual for a frozen LLM, cutting construction storage by 94.7% and raising mean token F1 from 32.83 % to 34.48 % on five knowledge-QA datasets[6][1].

[P5] QueryFlow: Multi-Hop Graph Retrieval with Adaptive Query Evolution · relevance 0.41
QueryFlow introduces an adaptive, recurrent query-evolution mechanism for multi-hop graph retrieval that markedly improves evidence recall and downstream answer quality on four QA benchmarks.

[P1] From Logs to Faults: Reference-Free Fault Localization for Retrieval-Augmented Generation Pipelines

Hira Naveed, Scott Barnett, Stefanus Kurniawan, John Grundy et al. · arXiv 2611.03725v1 · https://arxiv.org/abs/2611.03725v1 · relevance 0.47

TL;DR: RAGLog is a reference-free tool that localizes faults in Retrieval-Augmented Generation pipelines from execution logs, achieving pooled F1 ~ 0.73 on natural runs and ~ 0.84 on injected runs with low latency and cost.

Problem
Fault localization in RAG pipelines is manual, time-consuming, and difficult because faults can propagate across interdependent stages or remain hidden even when the final answer is correct [1]. Existing evaluation methods focus on single stages and require reference answers or code instrumentation, limiting their applicability [1].

Method
RAGLog converts heterogeneous API logs into a unified RAG schema and applies a hybrid classification that first uses rule-based matching for known APIs and falls back to an LLM for unseen calls [2][3]. Stage-level fault classifiers are trained offline using regularized logistic regression, shallow decision trees, and extremely randomized trees, with stage-specific probability thresholds chosen to keep false-positive rates <= 10 % [3]. The system combines rule-based, embedding-based, and LLM-based checks to flag faulty stages, and it can be applied to new pipelines without additional labeling [3][5].

Key results
The RAGLog Faults dataset contains 3,134 runs from five open-source pipelines, each run labeled with stage-level fault information [3][5].
- On natural runs, RAGLog with Claude Haiku 5.5 attains a pooled F1 of 0.73, outperforming Claude Code (0.58) and Codex (0.52) [1].
- On injected runs, pooled F1 rises to 0.84, compared with 0.80 for Claude Code and 0.82 for Codex [1].
- Stage-level F1 scores for RAGLogHaiku 5.5 are high for early stages (rewriter, embedder F1 ~ 0.86-0.97) and reach 0.69 for the generator, exceeding Claude Code (0.55) and Codex (0.50) on the same stage [6].
- Analysis time is about five seconds per run at a cost of roughly US $0.005, which is lower than the agent baselines in both latency and expense [1][5].

Limitations
- Some stages (e.g., embedder) have very few faults in the natural test set, making their individual F1 scores unstable and macro-F1 sensitive to outliers [7].
- Although development-set thresholds keep average false-positive rates below 10 %, specific stages can exceed this bound (e.g., generator FPR ~ 10.8 % on natural DragonBall runs) [7].
- The evaluation is limited to five open-source pipelines and a total of 3,134 runs, which may restrict generalizability to larger or proprietary systems [3][5].
- Variability of the agent baselines was not assessed due to their higher cost, leaving their run-to-run

stability unknown [7].

Sources

[1] arXiv 2611.03725v1.pdf, arXiv abstract

[2] arXiv 2611.03725v1.pdf, page 7

[3] arXiv 2611.03725v1.pdf, page 13

[5] arXiv 2611.03725v1.pdf, page 19

[6] arXiv 2611.03725v1.pdf, page 24

[7] arXiv 2611.03725v1.pdf, page 39

[P2] Reliability Stress Testing of RAG Retrieval under Query-Relevant Corpus Poisoning

Janet Sung, Kainan Zhou, Gangzhen Qian, Hang Xiao et al. · arXiv 2611.04264v1 · https://arxiv.org/abs/2611.04264v1 · relevance 0.46

TL;DR: Query-aware corpus poisoning can dramatically degrade RAG retrieval quality, achieving near-perfect target exposure while dropping nDCG@10 by up to 60% even with a tiny fraction of poisoned documents.

Problem
Retrieval-augmented generation (RAG) systems admit external documents into the model context only after a retriever ranks them, creating an admission step that attackers can exploit by inserting query-relevant poison documents into the corpus [1][2]. The study investigates how such corpus poisoning affects retrieval performance, independent of any downstream language-model generation [1][2].

Method
The authors stress the admission step by varying (i) document content (query-only, neutral, payload), (ii) attacker query information (0, 2, 4, or full query terms), (iii) per-target poison budget P ? {1,3,5}, and (iv) contamination scope (isolated target vs. multi-target) [3]. Experiments use 623 BEIR queries from SciFact and NFCorpus, measuring nDCG@10, recall, target exposure, any-poison exposure, and evidence-safety (SafeR) for BM25, LSA, Hybrid, and a pretrained dense retriever (BGE-small) [1][4][5][6].

Key results
- Adding five query-targeted documents per query reduces classical-retriever nDCG@10 by 40.2-61.5 % in isolated-target stress [1].
- For BM25 on SciFact, isolated P=5 drops nDCG@10 from 0.662 (clean) to 0.255, while Hybrid drops from 0.465 to 0.183; on NFCorpus, BM25 falls from 0.308 to 0.181 and Hybrid from 0.267 to 0.160 [4].
- Target exposure reaches 1.0 for Hybrid under isolated P=5 on SciFact and 0.910 on NFCorpus, with any-poison exposure also at 1.0 for SciFact and 0.910 for NFCorpus [4].
- Increasing P from 1 to 5 in the Hybrid isolated setting lowers SciFact nDCG@10 from 0.328 -> 0.183 (target exposure stays at 1.0) and NFCorpus nDCG@10 from 0.230 -> 0.160 (target exposure rises from 0.889 -> 0.910) [5].
- A pretrained BGE-small dense retriever attains 100 % target exposure with query-only poisoning (P=5) on both datasets; adding payload changes SciFact nDCG@10 only from 0.269 -> 0.271 and reduces NFCorpus exposure slightly to 98.8 % [1][6].
- On 162 long SciFact queries, two random terms (covering 19.9 % of eligible terms) yield 39.5 % Hybrid target exposure, while four terms (39.7 % coverage) yield 89.5 % exposure [1].

Limitations
The evaluation stops at the retrieval layer, so exposure does not equate to downstream attack success [7]. Attacker query information is modeled transparently rather than stealthily, and isolated low-fraction experiments still assume strong per-target targeting [7]. Neutral-control snippets do not perfectly match payload term frequency or semantics, and fixed two/four-term budgets are hard to interpret on short

queries [7]. High-IDF conditions assume access to clean-corpus statistics, and the dense-retriever validation covers only one compact model (BGE-small) without larger embeddings or production vector-search stacks [7]. BEIR relevance judgments are incomplete, affecting evidence-availability and SafeR metrics [7].

Sources

[1] arXiv 2611.04264v1.pdf, arXiv abstract

[2] arXiv 2611.04264v1.pdf, page 1

[3] arXiv 2611.04264v1.pdf, page 2

[4] arXiv 2611.04264v1.pdf, page 3

[5] arXiv 2611.04264v1.pdf, page 4

[6] arXiv 2611.04264v1.pdf, page 5

[7] arXiv 2611.04264v1.pdf, page 6

[P3] Enhancing Stock Market Information Retrieval with Retrieval-Augmented Generation

Huan Phong Thai, Thu Anh Ngoc Le, Nhu Huynh Thi Le, Nhan Huu Tran et al. · arXiv 2611.03747v1 · https://arxiv.org/abs/2611.03747v1 · relevance 0.43

TL;DR: The paper presents a Retrieval-Augmented Generation (RAG) system for stock-market queries that achieves high ROUGE scores and low latency, demonstrating promising accuracy and speed for financial information retrieval.

Problem
Traditional NLP models struggle with the dynamic, complex nature of financial data, often failing to provide real-time, contextually accurate, and explainable outputs for investors and analysts[2]. Efficient retrieval of relevant financial information and generation of accurate insights are therefore critical challenges in the stock-market domain[2].

Method
The proposed system combines a retriever module (encoding inputs, approximate nearest-neighbor search, and a datastore of external knowledge) with a generator module that synthesizes responses from retrieved content[3]. Retrieval is performed using FAISS embeddings followed by a BM25 re-ranking step to ensure both semantic similarity and keyword relevance[5]. The pipeline includes Vietnamese-specific preprocessing (normalization, tokenization with Underthesea, and stop-word removal) to improve text embeddings for downstream retrieval and generation[4]. Evaluation relies on ROUGE-1, ROUGE-2, and ROUGE-L metrics to compare generated answers with reference texts[6].

Key results
- The system attains "high" ROUGE-1, ROUGE-2, and ROUGE-L scores, indicating strong lexical and structural alignment with reference documents[1][7].
- Average response latency is 1.16 seconds per query, demonstrating timely processing suitable for real-time applications[1][7].
- No explicit numerical ROUGE values or baseline comparisons are reported in the excerpts[1].

Limitations
The paper does not provide detailed quantitative ROUGE results, baseline comparisons, or an explicit analysis of failure cases; limitations are therefore not fully disclosed[1]. The authors note future work to improve handling of complex financial queries, integrate additional knowledge sources, and further reduce latency, implying current constraints in query complexity and efficiency[7].

Sources

[1] arXiv 2611.03747v1.pdf, arXiv abstract

[2] arXiv 2611.03747v1.pdf, page 3

[3] arXiv 2611.03747v1.pdf, page 6

[4] arXiv 2611.03747v1.pdf, page 9

[5] arXiv 2611.03747v1.pdf, page 12

[6] arXiv 2611.03747v1.pdf, page 14

[7] arXiv 2611.03747v1.pdf, page 17

[P4] From Lookups to Lessons: Turning Retrieved Knowledge into Parametric Memory

Changjiang Han, Bowei He, Ye Yuan, Weixu Zhang et al. · arXiv 2611.00450v1 · https://arxiv.org/abs/2611.00450v1 · relevance 0.41

TL;DR: ResMem distills a retrieval-augmented teacher into an additive logit residual for a frozen LLM, cutting construction storage by 94.7% and raising mean token F1 from 32.83 % to 34.48 % on five knowledge-QA datasets[6][1].

Problem
Retrieval-augmented generation improves factual answering but incurs extra storage and inference costs, and existing parametric-memory approaches that simply mimic retriever outputs may not enhance the final output when combined with the LLM's own predictions[1].

Method
ResMem trains a parametric memory offline while keeping the base language model frozen; the memory follows the MLP Memory architecture but uses an LLM that reads retrieved text as an offline teacher to generate target distributions[1][2]. The student learns an additive logit residual that aligns its combined prediction with the teacher distribution, and inference adds the scaled residual to the base logits before the final softmax[3].

Key results
On a Wikipedia subset, ResMem reduces the storage needed for teacher construction by 94.7 % (~18.8×) compared with a kNN datastore[4]. Across five QA datasets, mean token F1 improves from 32.83 % (frozen LLM) to 34.48 % (ResMem)[1][6]. The F1 margin over MLP Memory grows from 0.32 points on open-domain QA to 1.77 points on multi-hop QA[4]. On the HaluEval hallucination-detection benchmark, accuracy rises from 54.83 % (Base) to 63.09 % (ResMem), surpassing MLP Memory's 60.34 %[4]. Inference adds about 8.6 % latency over the base model (~16 % at batch 1) while peak memory usage drops from 5.75 GiB to 2.88 GiB in BF16 mode[7].

Limitations
The current design reads only a single hidden state and writes solely to the output logits, so the teacher's gain is not propagated through the base model's intermediate layers[6].

Sources

[1] arXiv 2611.00450v1.pdf, arXiv abstract

[2] arXiv 2611.00450v1.pdf, page 3

[3] arXiv 2611.00450v1.pdf, page 4

[4] arXiv 2611.00450v1.pdf, page 6

[6] arXiv 2611.00450v1.pdf, page 9

[7] arXiv 2611.00450v1.pdf, page 14

[P5] QueryFlow: Multi-Hop Graph Retrieval with Adaptive Query Evolution

Manzong Huang, Chenyang Bu, Yi He, Jiapu Wang et al. · arXiv 2611.04625v1 · https://arxiv.org/abs/2611.04625v1 · relevance 0.41

TL;DR: QueryFlow introduces an adaptive, recurrent query-evolution mechanism for multi-hop graph retrieval that markedly improves evidence recall and downstream answer quality on four QA benchmarks.

Problem
Multi-hop retrieval-augmented generation must gather complementary evidence across multiple pages within a limited budget, but scoring evidence only against the original question can miss items whose relevance emerges only after intermediate facts are retrieved, and existing graph retrievers keep the query fixed while updating entity representations [1][2].

Method
QueryFlow treats evidence recovery as a coupled evolution of entity representations and a shared query state; at each layer the query state controls boundary inputs at question-linked entities and the relation features used for message passing, allowing accumulated graph context to reshape the retrieval condition [2][3]. The system builds an entity graph linking entities to documents, initializes the query from frozen text encodings, and iteratively propagates relevance through the graph while updating the query state recurrently [2][3].

Key results
- Across four multi-hop QA datasets, QueryFlow attains macro Recall@10 of 81.93 % and Full Recall@10 of 70.38 %, which are gains of 5.27 and 8.59 percentage points over a non-evolving baseline (? = 0) [4][6].
- The full model also reaches macro R@30 = 87.12 % and FR@30 = 77.78 % [4].
- Compared with representative baselines, QueryFlow improves macro LLM-Acc by 8.0 points (to 0.696) and Contain by 6.9 points (to 0.642) over LinearRAG, and by 11.6 and 5.6 points respectively over GFM-RAG [3][6].
- Gains are consistent across datasets; for example, LLM-Acc rises by 15.0 points on 2WikiMultiHopQA and by 7.0 points on both MuSiQue and ConcurrentQA [3].
- Ablation shows that removing query evolution drops FR@10 from 70.38 % to 61.78 %, and removing boundary feedback reduces FR@10 to 65.65 % [4].
- Efficiency analysis reports total retrieval + answer time of 0.580 ks, a 90.3 % reduction versus GFM-RAG, and token consumption of 3.59 M versus 5.44 M for GFM-RAG [5].

Limitations
The provided excerpts do not discuss any limitations of the approach, such as scalability to larger corpora, robustness to noisy graphs, or performance on non-QA tasks.

Sources

[1] arXiv 2611.04625v1.pdf, arXiv abstract

[2] arXiv 2611.04625v1.pdf, page 3

[3] arXiv 2611.04625v1.pdf, page 5

[4] arXiv 2611.04625v1.pdf, page 7

[5] arXiv 2611.04625v1.pdf, page 8

[6] arXiv 2611.04625v1.pdf, page 9